Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 1

- Reading:
  - Read the syllabus. Ask questions, report errors/typos.
  - Read Chap. 1 of CLRS (the textbook)
  - Read Chap. 2 in preparation for next lecture

Today's topics

- Syllabus items
- Brief introduction to algorithms
  - What is an algorithm?
  - Examples of algorithms
- Sorting problem
- Bubblesort

SYLLABUS ITEMS

Course Mechanics

- Type of course
  - theoretical material
  - problem sets
  - no required programming
    - but you're encouraged to experiment with writing code for the algorithms
  - emphasis on analyzing things: proving algorithms correct, analyzing running times, uses mathematical tools
- Prerequisites
  - CS-UY 1134 Data Structures and Algorithms (C- or better)
  - MA-UY 2314 Discrete Mathematics
  - Assume you'll remember: Basic data structures, asymptotic notation, and basic probability theory
  - [Check with me if you haven't taken the prerequisite courses or their equivalents at NYU-SH or NYU-AD]
- Concurrent Registration
  - EX-UY 1 Examination Hour

Communication

- Ed Discussion will be used for announcements, discussion, and posting course and lecture info.
- Gradescope will be used for homeworks

Homework

- Essential
  - Need to do as preparation for the exams
  - Don't just learn by watching, learn by doing
  - Like learning to play basketball, or the piano
- Homeworks approx every 2 weeks
  - Important to do homework learn material and practice for the exam
  - Get pass/fail on each homework
  - Doesn't directly affect your grade, but can affect how your grade is computed (see next slide)
- Homework rules will be posted on EdStem, but briefly:
  - Can work with one or two other people
  - Need to handwrite your answers
  - Generative AI can only be used to get ideas for solving problems, not to generate answers

Participation

- Attendance in class is expected
- Participation is 2% of the grade
  - Most participation points will come from surveys in class that you will do on your phone.
  - Others will be from answering questions outside class on Gradescope
- Be engaged!
  - Ask questions in class
  - Come to office hours
  - Participate in discussions on Ed Discussion
  - Work with study partners

Exams

- **Midterm 1** (Tues, October 27)
- **Midterm 2** (Tues, November 24)
- **Final exam** (during Finals week Dec 16 through 22)
  - Do not make travel plans before end of this period
- No makeup exams without documented excused absence per Tandon policy (see syllabus)

Grade percentages

- Different formula depending on doing homework
- If pass all homework, or fail only one homework, your score will be highest of the following 3 alternatives:
  - (i) Exam 1 (32%), Exam 2 (33%), Final (33%), Participation (2%)
  - (ii) Exam 1 (30%), Exam 2 (18%), Final (50%), Participation (2%)
  - (iii) Exam 1(18%), Exam 2 (30%), Final (50%), Participation (2%)
- If fail two or more homework, your score will be computed according to scheme (i) above

Textbook

- Required textbook:
  - Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein
  - **4th Edition**
  - aka CLRS
- See syllabus for recommendations on other textbooks, lecture notes, and videos, if interested

Academic Integrity

- NYU Tandon Code of Student Conduct
- Don't cheat!
- Exam policy will specify what is allowed and not allowed
  - Penalties for cheating likely to be F in the course and report to dept and Tandon.
- Homework policy will specify what is allowed and not allowed on HWs
- If you have a question about what's allowed, ask Prof. Hellerstein

COURSE TOPICS

Course Topics

- Analysis of algorithms
  - Does the algorithm work correctly?
  - What is the running time of the algorithm?
  - Methods of analysis
- Design of algorithms
  - Toolbox of algorithmic techniques
    - Iterative algorithms
    - Greedy algorithms
    - Divide and conquer
    - Randomization
    - Dynamic Programming

Course Topics (continued)

- Fundamental algorithms
  - Sorting
  - Searching
  - Graph algorithms
- Computational hardness
  - P vs. NP
- Selected other topics as time permits

Goals of Course

- Main focus of the course is for you to learn:
  - Well-known useful algorithms
  - Techniques for designing algorithms for new problems
  - Algorithm analysis
    - Is the algorithm correct?
    - Is it efficient (fast)?

  - Brief introduction to computational complexity / hardness:
    - If you have an algorithm for solving a problem, could there be another algorithm that is more efficient?
    - If you can't find any efficient algorithm for solving a problem, is it because there is no efficient algorithm?

Introduction to algorithms

Algorithm

- Problem to solve
  - In this course, problems will be precisely defined
- Algorithm for solving problem
  - Step-by-step description of how to solve the problem

- People have been designing algorithms to solve computational problems since ancient times
  - algorithms for multiplication, factorization, square roots, solving systems of linear equations…
  - al-Khwarizmi (9th century)
    - wrote treatises describing algorithms for solving algebra and arithmetic problems
    - word “algorithm” comes from his name
- Modern study of algorithms begins in 20th century with development of computers

Algorithms are everywhere

- Google search
- MTA Trip Planner
- Medical imaging
- Video games
- …

Sorting

Question

- Is list 2, 3, 5, 5, 7, 9 sorted in
  - Increasing order? Potentially ambiguous Yes
  - Ascending order? Yes
  - ~~Strictly increasing order?~~ A few
  - Non-decreasing order? Yes

Textbook

- Problem definition
  - Informal
    - Sort numbers in a sequence (list) so they appear in increasing order
  - More formally
    - Input: sequence of n numbers $< a_1, a_2, \ldots, a_n >$
    - Output: permutation (reordering)
      $< a'_1, a'_2, \ldots, a'_n >$ of items in input sequence such that
      $a'_1 \leq a'_2 \leq \ldots \leq a'_n$

- <3, 16, 5, 25> 3, 5, 16, 25
- <3, 16, 5, 25, 5> 3, 5, 5, 16, 25
- Can also define sorting problem for other types of data
  - e.g. names
    alphabetical order

Example Sorting Algorithm

Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

A is array of length n, stores the input sequence
(indexed starting at 1)



Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

1 2 3 4

A = [7, 16, 5, 25] n = 4

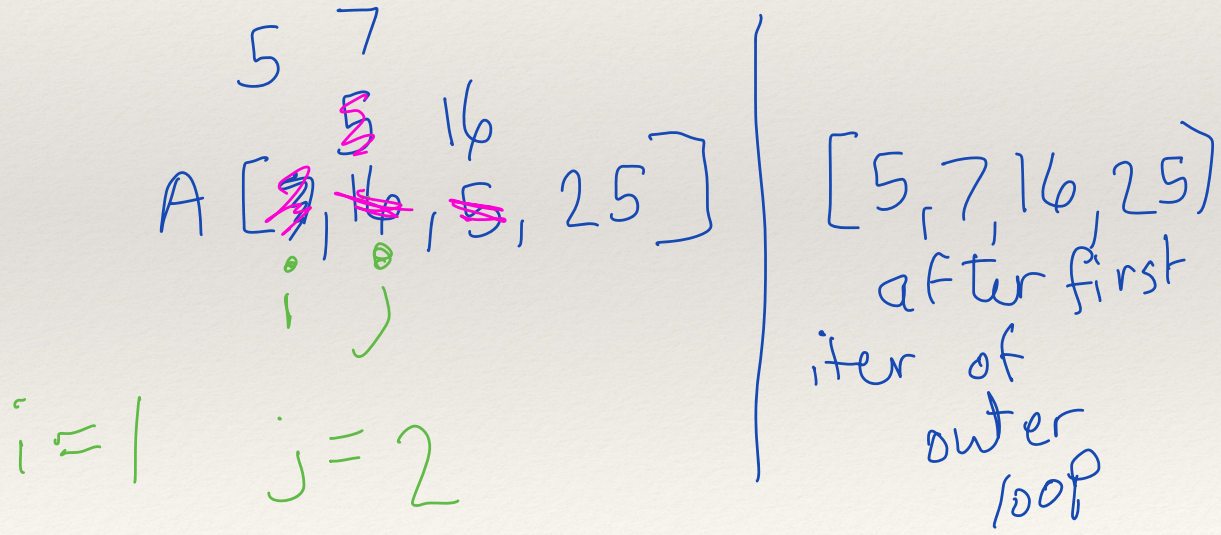
Why is it called Bubblesort?

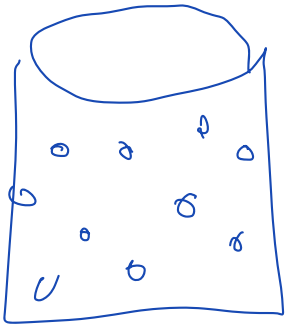
Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

A = [7, 16, 5, 25]

Why is it called Bubblesort?





(5)



Top of glass

Bottom of glass

Correctness of Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Proof of correctness of Bubblesort?

Need to show A is sorted when Bubblesort terminates

To make things easier, assume all elements of A are distinct

Comment on Notation

1 2 3
A = [blue, red, green]
A[1] = blue

- The book doesn't use the same conventions as Python
  - The book indexes arrays starting from 1 unless otherwise specified
  - When it specifies a range A[i:j] it means all entries from A[i] up to **and including** A[j]

A[1:2] = {blue, red}

- We will use the same conventions as in the book

Correctness of Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Proof of correctness of Bubblesort?

Need to show A is sorted when Bubblesort terminates

To make things easier, assume all elements of A are distinct

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Proof Idea: Show that Bubblesort is making progress with each iteration of outer loop.

Prove loop invariant, property that is always true at start of each iteration of the loop.

What property does A have at the start of each iteration of outer loop?

A[1 : i-1] contains the smallest i-1 elements in array in sorted orde


i - 1
i

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Proof Idea: Show that Bubblesort is making progress with each loop iteration.

Prove loop invariant, property that is always true at start of each iteration of the loop.

What property does A have at the start of each iteration of outer loop?
$A[1 : i - 1]$ contains smallest $i - 1$ elements of A, in sorted order

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Loop invariant: At start of each iteration of the outer loop,
$A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest

How to prove loop invariant holds?

1. Show true before first iteration of loop.

2. Show if it's true at start of an iteration, it's still true at start of next iteration.

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Loop invariant: At start of each iteration of the outer loop,
$A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest

How to prove loop invariant holds?

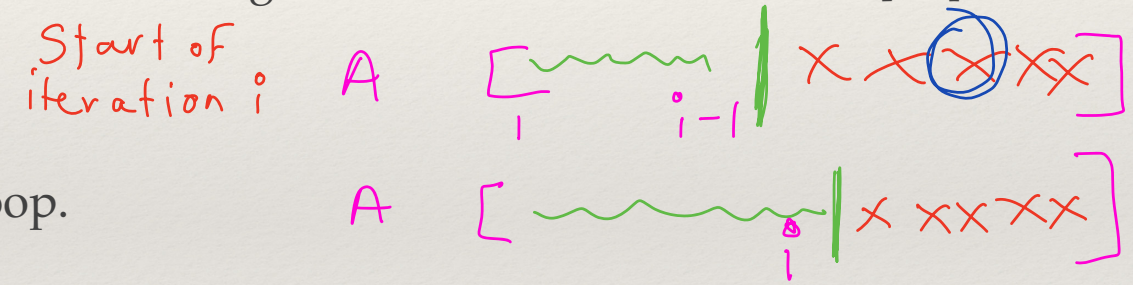
1. Show true before first iteration of loop.

Trivial

2. Show if it's true at start of an iteration, it's still true at start of next iteration.

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Loop invariant: At start of each iteration of the outer loop,
$A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest



How to prove loop invariant holds?

1. Show true before first iteration of loop.

Trivial

2. Show if it's true at start of an iteration, it's still true at start of next iteration.

Consider what happens to ith smallest element during iteration i

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Loop invariant: At start of each iteration of the outer loop,
$A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest

How to prove loop invariant holds?

1. Show true before first iteration of loop.

Trivial

2. Show if it's true at start of an iteration, it's still true at start of next iteration.

Consider what happens to ith smallest element in iteration i

Once index j reaches it, ith smallest element continues getting swapped until it settles in position i

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Loop invariant: At start of each iteration of the outer loop,
$A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest

How to prove loop invariant holds?
1. Show true before first iteration of loop.
Trivial
2. Show if it's true at start of an iteration, it's still true at start of next iteration.
Consider what happens to ith smallest element in iteration i
Once index j reaches it, ith smallest element continues getting swapped until it settles in position i
3. Having shown loop invariant holds, show that loop terminates.

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```



Loop invariant: At start of each iteration of the outer loop, $A[1 : i-1]$ contains $i-1$ smallest elements of original A, in sorted order, and A[i:n] contains the rest

At termination (i=n), what does loop invariant imply?

A[1:n-1] contains the n-1 smallest elements in A, in sorted order, A[n:n] contains rest

⇒ A is sorted b/c A[n] must contain max element of A.

Performance of Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Could consider various aspects of performance like….

Performance of Bubble Sort

```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Could consider various aspects of performance like
running time, space (memory), how much battery, how much data transfer between main and secondary storage…

In this course, main focus is on running time, sometimes space

Running time

- Actual running time of algorithm depends on many factors
  - Programming language

Running time

- Actual running time of algorithm depends on many factors
  - Programming language
  - Compiler version
  - Optimization settings
  - System load
  - Hardware
  - ….

Running time

- Take more conceptual approach
- Count number of "basic" operations:
  - store/load value
  - index arithmetic
  - arithmetic operations: +, *, -, /
- We'll express running times in asymptotic notation (big Oh) so we won't worry e.g., whether to count i++ as one or two operations

